CNNs: generalização e interpretabilidade

Rafael Stutz

Aula 5 · 19/10/2026

Deep Learning (DEPAI) · IASEG

Roteiro

1. O que a CNN aprendeu: mapas de características;
2. Sobreajuste e regularização: *`EarlyStopping`*, Dropout e L2;
3. Aumentação de dados: quando ajuda e quando atrapalha;
4. Transferência de aprendizado com ResNet50 no Keras;
5. Interpretabilidade: mapas de ativação, Grad-CAM e limites;
6. Estudos de caso e aplicação no projeto.

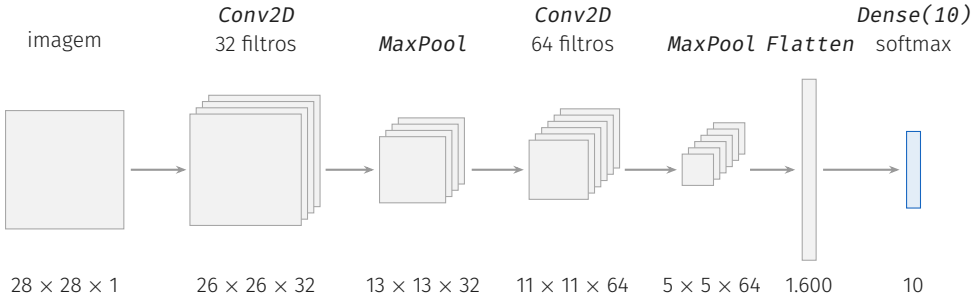
Abertura

Onde estamos

Aula 1 · 05/10 O curso; história do deep learning; redes neurais	**Aula 2** · 06/10 Revisão; viés, variância e ruído; atalhos e vazamento; Keras	**Aula 3** · 07/10 Funções de ativação e de perda; taxa de aprendizado, lote e época	**Aula 4** · 08/10 Imagens, convoluções, pooling e CNN no Keras
Aula 5 · 19/10 Regularização, aumentação, transferência e Grad-CAM	**Aula 6** · 20/10 Séries temporais: janelas, referências, RNN e LSTM	**Aula 7** · 21/10 Texto: embeddings, atenção, Transformers e LLMs	**Entrega** · 22/10 Apresentação do projeto e perguntas

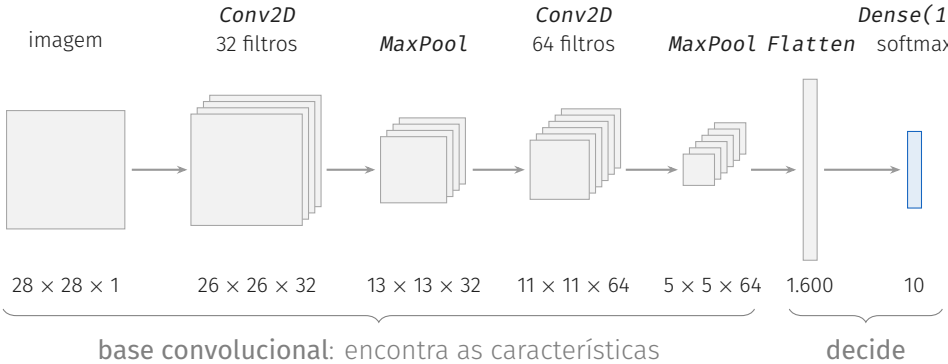
Na Aula 4, aprendemos a montar uma **CNN**. Hoje, vamos perguntar: como fazer essa rede funcionar melhor em imagens novas e investigar suas previsões?

Relembrando a estrutura da CNN



A imagem **encolhe** (28, 26, 13, 11, 5) e a pilha de mapas **cresce** (1, 32, 64): menos “onde”, mais “o quê”.

Relembrando a estrutura da CNN



A imagem **encolhe** (28, 26, 13, 11, 5) e a pilha de mapas **cresce** (1, 32, 64): menos "onde", mais "o quê". No fim, `Flatten` e uma camada densa com softmax: a classificação da Aula 3.

Da Aula 4 para a Aula 5

Já sabemos

- Imagens são números: altura × largura × canais;
- Filtros aprendem padrões locais e geram mapas;
- *Conv2D*, pooling, *Flatten*, *Dense*;
- Montar, treinar e avaliar uma CNN no Keras.

A pergunta de hoje

A CNN funciona no **treino**.

E se tivermos poucas imagens, outra câmera ou uma previsão suspeita?

Para discutir

Se a rede acertou 100% das imagens de treino, isso garante que acertará fotos novas?

O que a rede aprende

A nossa CNN olhando um 7



- Primeira camada: os mapas ainda se parecem com o 7, e cada um destaca um pedaço;

6 dos 32 e 6 dos 64 mapas da CNN que treinamos. Escuro: valor alto.

A nossa CNN olhando um 7



- Primeira camada: os mapas ainda se parecem com o 7, e cada um destaca um pedaço;
- Segunda camada: mapas menores, e cada um fica com **uma parte**: o traço de cima, a diagonal, o canto do 7;

6 dos 32 e 6 dos 64 mapas da CNN que treinamos. Escuro: valor alto.

A nossa CNN olhando um 7



- Primeira camada: os mapas ainda se parecem com o 7, e cada um destaca um pedaço;
- Segunda camada: mapas menores, e cada um fica com **uma parte**: o traço de cima, a diagonal, o canto do 7;
- No fim, a softmax deu mais de 99,9% na classe “7” (saída da softmax).

6 dos 32 e 6 dos 64 mapas da CNN que treinamos. Escuro: valor alto.

O que cada camada vê: uma ResNet olhando a placa

camada 1 | bloco 2 | bloco 3 | bloco 4



$120 \times 250 \times 64$

Mais funda, **menor o mapa** e **mais mapas**: cada pixel resume um pedaço maior da placa.

Mapas reais de uma ResNet50 treinada no ImageNet. Dois mapas por profundidade.

O que cada camada vê: uma ResNet olhando a placa

camada 1 | bloco 2 | bloco 3 | bloco 4



120 × 250 × 64 | 60 × 125 × 256

Mais funda, **menor o mapa** e **mais mapas**: cada pixel resume um pedaço maior da placa.

Mapas reais de uma ResNet50 treinada no ImageNet. Dois mapas por profundidade.

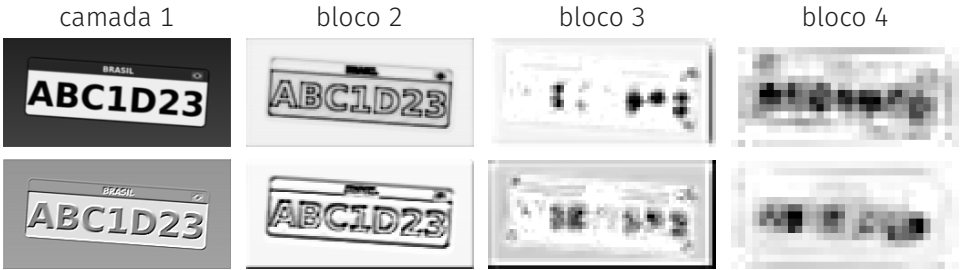
O que cada camada vê: uma ResNet olhando a placa



Mais funda, **menor o mapa** e **mais mapas**: cada pixel resume um pedaço maior da placa.

Mapas reais de uma ResNet50 treinada no ImageNet. Dois mapas por profundidade.

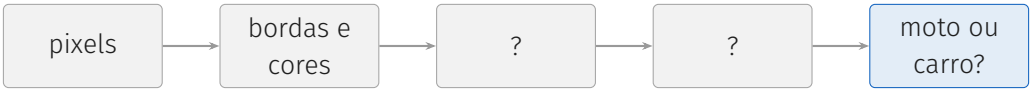
O que cada camada vê: uma ResNet olhando a placa



Mais funda, **menor o mapa** e **mais mapas**: cada pixel resume um pedaço maior da placa. As primeiras camadas ainda "desenham" a placa; as últimas marcam regiões, e já não se parecem com a foto.

Mapas reais de uma ResNet50 treinada no ImageNet. Dois mapas por profundidade.

Para discutir: as camadas do meio

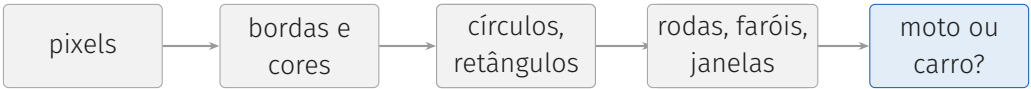


Para discutir

Uma CNN classifica o tipo de veículo nas câmeras. O que vocês esperam que as camadas do meio detectem?

Esquema ilustrativo: as camadas de uma rede real não aprendem exatamente essas partes.

Para discutir: as camadas do meio



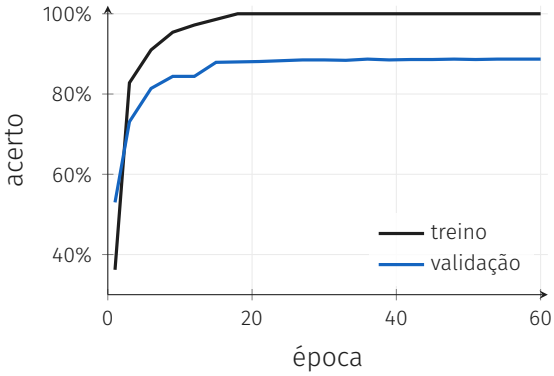
Para discutir

Uma CNN classifica o tipo de veículo nas câmeras. O que vocês esperam que as camadas do meio detectem?

Formas simples, e depois **partes**: rodas, faróis, janelas, guidão. Ninguém diz isso à rede: é o que costuma aparecer quando se olham os mapas de redes treinadas (Zeiler e Fergus, 2014). Como na placa da Aula 1, cada camada monta algo mais abstrato.

Esquema ilustrativo: as camadas de uma rede real não aprendem exatamente essas partes.

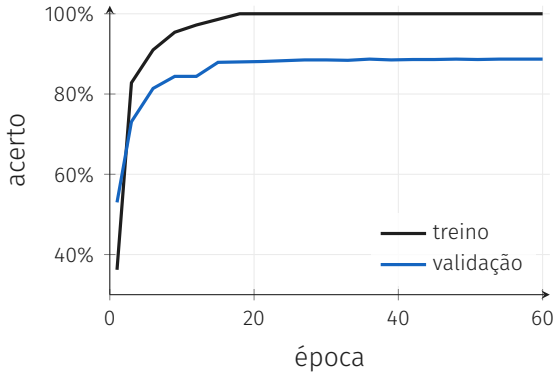
Poucas imagens: o sobreajuste volta



- A mesma CNN, treinada com só **500 dígitos**, por 60 épocas;

Experimento: 500 treino; dos 10.000 testes oficiais, 2.000 para validação e 8.000 para teste final.

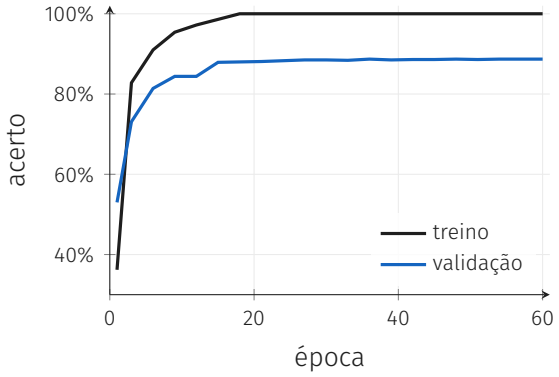
Poucas imagens: o sobreajuste volta



- A mesma CNN, treinada com só **500 dígitos**, por 60 épocas;
- Treino: **100%** a partir da época 18. A rede se ajustou demais às 500 imagens;
- Validação: para em 88,7%;

Experimento: 500 treino; dos 10.000 testes oficiais, 2.000 para validação e 8.000 para teste final.

Poucas imagens: o sobreajuste volta



- A mesma CNN, treinada com só **500 dígitos**, por 60 épocas;
- Treino: **100%** a partir da época 18. A rede se ajustou demais às 500 imagens;
- Validação: para em 88,7%;
- É o sobreajuste da Aula 2. Com imagens, poucos exemplos é a situação mais comum: fotografar e rotular custa caro.

Experimento: 500 treino; dos 10.000 testes oficiais, 2.000 para validação e 8.000 para teste final.

Sobreajuste e regularização

A rede aprendeu ou apenas se ajustou ao treino?



Para discutir

O modelo fez muito bem a tarefa nos exemplos vistos. Por que o resultado caiu nos exemplos novos?

Valores do experimento com 500 dígitos apresentado no slide anterior.

A rede aprendeu ou apenas se ajustou ao treino?



Para discutir

O modelo fez muito bem a tarefa nos exemplos vistos. Por que o resultado caiu nos exemplos novos?

O problema é sobreajuste: a rede se adapta demais às particularidades do treino.
Regularizar significa usar estratégias para favorecer a generalização.

Valores do experimento com 500 dígitos apresentado no slide anterior.

Regularização: três maneiras de ajudar

Parar a tempo *EarlyStopping*	Desligar unidades *Dropout*	Penalizar pesos L2

- **EarlyStopping**: interrompe o treino quando a validação deixa de melhorar;

Regularização: três maneiras de ajudar

Parar a tempo *EarlyStopping*	Desligar unidades *Dropout*	Penalizar pesos L2

- **EarlyStopping**: interrompe o treino quando a validação deixa de melhorar;
- **Dropout**: desativa aleatoriamente algumas saídas durante o treino;

Regularização: três maneiras de ajudar

Parar a tempo *EarlyStopping*	Desligar unidades *Dropout*	Penalizar pesos L2

- **EarlyStopping**: interrompe o treino quando a validação deixa de melhorar;
- **Dropout**: desativa aleatoriamente algumas saídas durante o treino;
- **L2**: desestimula pesos de valor muito alto.

Regularização: três maneiras de ajudar

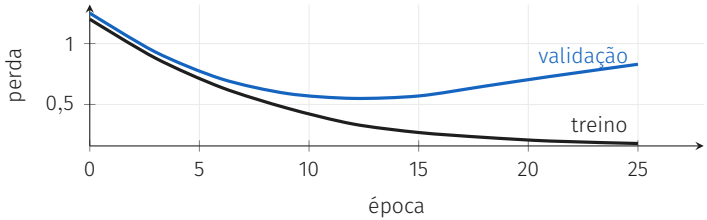
Parar a tempo *EarlyStopping*	Desligar unidades *Dropout*	Penalizar pesos L2

- **EarlyStopping**: interrompe o treino quando a validação deixa de melhorar;
- **Dropout**: desativa aleatoriamente algumas saídas durante o treino;
- **L2**: desestimula pesos de valor muito alto.

Atenção

Não são receitas: regularização demais pode fazer a rede aprender de menos.

EarlyStopping: quando interromper?

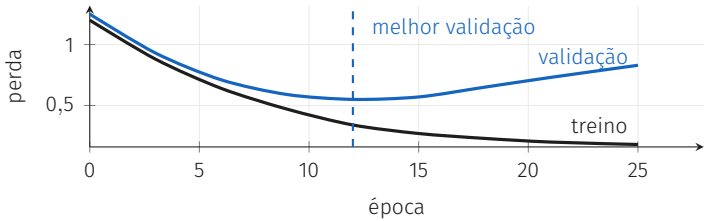


Curvas esquemáticas; não são resultados do experimento.

Palpite, antes de ver o resultado

O treino melhora até a época 25. Devemos usar os pesos da **última época**?

EarlyStopping: quando interromper?



Curvas esquemáticas; não são resultados do experimento.

Palpite, antes de ver o resultado

O treino melhora até a época 25. Devemos usar os pesos da **última época**?

Não necessariamente. A validação foi melhor na época 12. Continuar treinando pode piorar a generalização.

Pergunta: parar na primeira piora?

Para discutir

A melhor `val_loss` foi na época 12. Ela piorou nas três épocas seguintes. Com `patience=3`, quando paramos?

Época	12	13	14	15
val_loss	0,55	0,56	0,58	0,60

a) Na época 12, quando a validação foi melhor;

b) Na época 13, na primeira piora;

c) Na época 15, após três épocas sem melhora;

d) Apenas ao atingir o máximo de épocas.

Pergunta: parar na primeira piora?

Para discutir

A melhor `val_loss` foi na época 12. Ela piorou nas três épocas seguintes. Com `patience=3`, quando paramos?

Época	12	13	14	15
val_loss	0,55	0,56	0,58	0,60

Resposta: c

Paramos na **época 15**, após três épocas sem melhora. Se a validação voltasse a melhorar o suficiente, a espera recomeçaria.

Com `restore_best_weights=True`, recuperamos os pesos da **época 12**.

No Keras: épocas máximas e paciência

```
parar = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=3,
    min_delta=0.001,
    restore_best_weights=True
)

modelo.fit(
    X_train, y_train, epochs=50,
    validation_data=(X_val, y_val),
    callbacks=[parar]
)
```

Para discutir

Com *epochs=50*, o treino parou na época 18. Houve erro?

No Keras: épocas máximas e paciência

```
parar = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=3,
    min_delta=0.001,
    restore_best_weights=True
)

modelo.fit(
    X_train, y_train, epochs=50,
    validation_data=(X_val, y_val),
    callbacks=[parar]
)
```

Para discutir

Com *epochs=50*, o treino parou na época 18. Houve erro?

Não. 50 é o máximo; o callback pode parar antes.

Pergunta: qual paciência escolher?

Para discutir

No modelo A, a perda de validação cai de forma estável.

No modelo B, ela **oscila bastante**.

Qual opção faz mais sentido?

a) Usar sempre *`patience=1`*;
b) Considerar uma paciência maior para B;
c) Usar paciência menor para B;
d) Monitorar a perda de treino, por ser mais estável.

Pergunta: qual paciência escolher?

Para discutir

No modelo A, a perda de validação cai de forma estável.

No modelo B, ela **oscila bastante**.

Qual opção faz mais sentido?

Resposta: b

Uma piora isolada pode ser temporária. A paciência maior permite esperar pela próxima melhora.

- **Não existe valor universal**: 3 a 5 é só um começo;
- Considere a oscilação da validação e o custo do treino;
- Paciência alta não corrige validação ruim.

Pergunta: devemos usar EarlyStopping sempre?

Para discutir

Um colega afirma:

“Com *`patience=2`* em todos os modelos, nunca terei sobreajuste.”

O que está errado?

a) Nada: EarlyStopping impede todo sobreajuste;
b) O valor da paciência deve ser sempre 10;
c) EarlyStopping ajuda, mas não garante generalização;
d) EarlyStopping só pode ser usado com CNNs.

Pergunta: devemos usar EarlyStopping sempre?

Para discutir

Um colega afirma:

“Com *patience=2* em todos os modelos, nunca terei sobreajuste.”

O que está errado?

Resposta: c

O método pode reduzir o sobreajuste, mas depende de uma **validação representativa** e pode parar cedo demais.

- O teste não deve ser usado para decidir quando parar;
- A melhor época depende dos dados e do treinamento;
- É uma ferramenta útil, **não uma garantia**.

Pergunta: como escolher o número de épocas?

Para discutir

Colocamos *`epochs=20`*. Na época 20, a perda de validação **ainda está diminuindo**.

O que podemos fazer?

a) Sempre reduzir o limite para 10;
b) Considerar aumentar o máximo de épocas;
c) Usar as imagens de teste para escolher o limite;
d) Assumir que o modelo já convergiu.

Pergunta: como escolher o número de épocas?

Para discutir

Colocamos *epochs=20*. Na época 20, a perda de validação **ainda está diminuindo**.

O que podemos fazer?

Resposta: b

Se a validação continua melhorando, o modelo pode se beneficiar de mais treinamento.

- *epochs* define um **limite**, não a época ideal;
- EarlyStopping pode interromper antes do limite;
- Decisões de ajuste usam validação, **não teste**.

Outra forma de regularização: Dropout

Para discutir

Mesmo escolhendo bem o número de épocas, a rede ainda pode ficar **dependente demais** de algumas unidades da camada oculta.

O que poderíamos fazer para reduzir essa dependência?

a) Treinar sempre por mais épocas;
b) Desligar aleatoriamente algumas unidades durante o treino;
c) Usar o conjunto de teste para escolher a melhor época;
d) Congelar a camada oculta.

Outra forma de regularização: Dropout

Para discutir

Mesmo escolhendo bem o número de épocas, a rede ainda pode ficar **dependente demais** de algumas unidades da camada oculta.

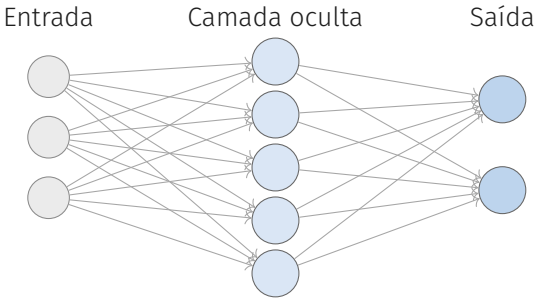
O que poderíamos fazer para reduzir essa dependência?

Resposta: b

Dropout desliga temporariamente algumas unidades durante o treino.

Assim, a rede não pode depender sempre das mesmas saídas.

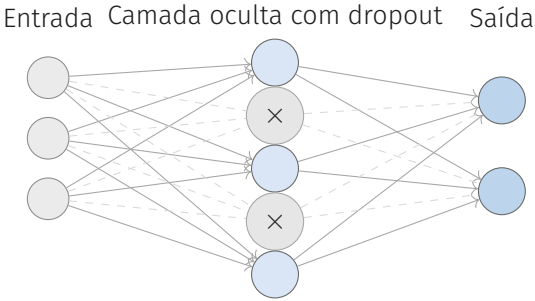
Sem dropout: todas as unidades participam



Sem dropout: todas as unidades da camada oculta participam daquele passo de treino.

- A rede usa **todas** as unidades da camada oculta;
- O risco é a rede passar a depender **demais** de algumas saídas.

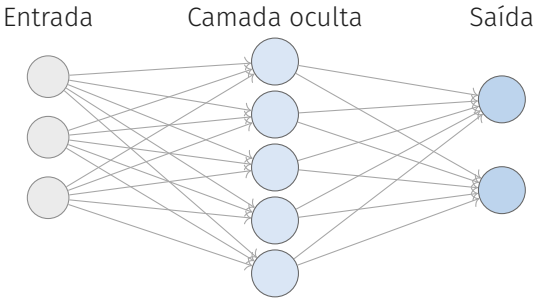
Com dropout no treino: algumas unidades são desligadas



Neste passo de treino, duas unidades foram desligadas.
As outras precisam aprender a resolver o problema sem depender delas.

- `Dropout(0.3)`: cada saída tem 30% de chance de ser zerada durante o treino;
- Isso dificulta que a rede fique **dependente demais** de poucos neurônios.

Na validação e na previsão: todas as unidades voltam



Em `evaluate` e `predict`, não há sorteio:
o dropout fica desligado e todas as unidades ficam ativas.

- O sorteio acontece **só no treino**;
- Na hora de validar ou prever, a rede usa a camada completa.

Pergunta: o Dropout zera os pesos?

Para discutir

Durante o treinamento, uma unidade foi desligada pelo Dropout.

O que aconteceu com os pesos aprendidos por essa unidade?

a) Foram apagados permanentemente;
b) Foram substituídos por novos pesos;
c) Foram preservados, mas a saída da unidade foi zerada nessa passagem;
d) Todos os pesos da rede ficaram congelados.

Pergunta: o Dropout zera os pesos?

Para discutir

Durante o treinamento, uma unidade foi desligada pelo Dropout.

O que aconteceu com os pesos aprendidos por essa unidade?

Resposta: c

O Dropout **não apaga os pesos**. Ele zera temporariamente a saída de algumas unidades.

- Os pesos continuam armazenados;
- A saída zerada não contribui para a previsão nessa passagem;
- A unidade poderá participar normalmente de outras passagens.

Pergunta: quanto maior o Dropout, melhor?

Para discutir

Uma CNN apresentava sobreajuste. Aumentamos o Dropout de 0.2 para 0.8.

Agora a rede erra **mais no treino e na validação**. O que pode ter acontecido?

a) O Dropout ficou forte demais;
b) O modelo necessariamente passou a generalizar melhor;
c) Devemos aumentar ainda mais o Dropout;
d) O Dropout só funciona com imagens coloridas.

Pergunta: quanto maior o Dropout, melhor?

Para discutir

Uma CNN apresentava sobreajuste. Aumentamos o Dropout de 0.2 para 0.8.

Agora a rede erra **mais no treino e na validação**. O que pode ter acontecido?

Resposta: a

Regularização excessiva pode prejudicar o aprendizado até dos padrões importantes.

- Dropout demais pode causar **subajuste**;
- Mais regularização não significa necessariamente melhor generalização;
- A intensidade deve ser avaliada usando os resultados de validação.

No Keras: adicionando Dropout à CNN

```
import keras
from keras import layers

modelo = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),  # NOVO
    layers.Dense(10, activation="softmax")
])

modelo.compile(optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

No Keras: mesmo EarlyStopping, novo modelo

```
parar = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=3,
    min_delta=0.001,
    restore_best_weights=True
)

modelo.fit(
    X_train, y_train, epochs=50,
    validation_data=(X_val, y_val),
    callbacks=[parar]
)
```

- **Dropout**: zera saídas durante o treino;
- **EarlyStopping**: decide quando interromper.

Os pesos também influenciam a generalização

Relembrando

A rede aprende **pesos**: números que controlam a influência dos sinais recebidos pelos neurônios.

Exemplo simplificado: **saída = entrada $\times$ peso**



Se x aumenta 0,1,
y aumenta 0,1.



Se x aumenta 0,1,
y aumenta 1.

Pergunta: qual das duas relações é mais sensível à mudança?

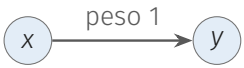
Isso não significa que todo peso grande seja inadequado.

Os pesos também influenciam a generalização

Relembrando

A rede aprende **pesos**: números que controlam a influência dos sinais recebidos pelos neurônios.

Exemplo simplificado: **saída = entrada $\times$ peso**



Se x aumenta 0,1,
y aumenta 0,1.



Se x aumenta 0,1,
y aumenta 1.

A da direita. Em situações assim, um peso grande amplifica pequenas mudanças.

Isso não significa que todo peso grande seja inadequado.

Uma nova estratégia: controlar os pesos

O problema

Alguns modelos podem se ajustar muito bem ao treino, mas usar pesos de grande magnitude que não são necessários para obter previsões semelhantes.

Uma ideia: além de cobrar pelos erros, cobrar também por pesos muito grandes.

$$\underbrace{\text{Perda total}}_{\text{o que a rede minimiza}} = \underbrace{\text{Erro}}_{\text{previsões}} + \underbrace{\text{Penalidade}}_{\text{pesos grandes}}$$

Uma nova estratégia: controlar os pesos

O problema

Alguns modelos podem se ajustar muito bem ao treino, mas usar pesos de grande magnitude que não são necessários para obter previsões semelhantes.

Uma ideia: além de cobrar pelos erros, cobrar também por pesos muito grandes.

$$\underbrace{\text{Perda total}}_{\text{o que a rede minimiza}} = \underbrace{\text{Erro}}_{\text{previsões}} + \underbrace{\text{Penalidade}}_{\text{pesos grandes}}$$

Isso se chama regularização L2

Ela acrescenta à perda uma penalidade que cresce com o **quadrado dos pesos**. Assim, incentiva o modelo a evitar pesos grandes sem necessidade.

L2: como funciona essa penalidade?

Regra

Elevamos os pesos ao quadrado, somamos os resultados e multiplicamos por um número que controla a força da penalidade.

$$\text{Penalidade L2} = \underbrace{\lambda}_{\text{força}} \times \underbrace{(w_1^2 + w_2^2 + \cdots)}_{\text{soma dos pesos ao quadrado}}$$

	Pesos	Soma dos quadrados
Modelo A	3 e 4	$3^2 + 4^2 = 25$
Modelo B	1 e 1	$1^2 + 1^2 = 2$

w_1, w_2 são pesos; λ controla a força da penalidade.

Pergunta: com o mesmo λ, qual modelo recebe maior penalidade?

L2: como funciona essa penalidade?

Regra

Elevamos os pesos ao quadrado, somamos os resultados e multiplicamos por um número que controla a força da penalidade.

$$\text{Penalidade L2} = \underbrace{\lambda}_{\text{força}} \times \underbrace{(w_1^2 + w_2^2 + \cdots)}_{\text{soma dos pesos ao quadrado}}$$

	Pesos	Soma dos quadrados
Modelo A	3 e 4	$3^2 + 4^2 = 25$
Modelo B	1 e 1	$1^2 + 1^2 = 2$

w_1, w_2 são pesos; λ controla a força da penalidade.

Resposta: A. Quanto maiores os quadrados dos pesos, maior a penalidade.

No Keras: acrescentando L2 à mesma CNN

Antes: a camada Dense que já conhecemos.

```
layers.Dense(64, activation="relu"),
```

Agora: importe *regularizers* e altere apenas essa Dense.

```
from keras import regularizers

layers.Dense(64, activation="relu",
    kernel_regularizer=regularizers.l2(1e-4)),
```

- *Dropout(0.3)* e o *EarlyStopping* do exemplo anterior **continuam iguais**;
- A penalidade entra na *loss* de treino **e** de validação; a *accuracy* não a inclui.

Pergunta: o que acontece se aumentarmos L2?

Para discutir

Trocamos *`regularizers.l2(1e-4)`* por *`regularizers.l2(1e-2)`*. O que mudou?

a) A penalidade dos pesos ficou mais forte;
b) A rede passou a desligar mais neurônios;
c) O treino terá obrigatoriamente mais acertos;
d) Os pesos da camada foram zerados.

Pergunta: o que acontece se aumentarmos L2?

Para discutir

Trocamos *`regularizers.l2(1e-4)`* por *`regularizers.l2(1e-2)`*. O que mudou?

Resposta: a

Demos **mais importância à penalidade** na função de perda.

Isso pode ajudar em alguns problemas, mas também dificultar o aprendizado se for excessivo.

Pergunta: L2 faz a mesma coisa que Dropout?

Para discutir

As duas técnicas são usadas para regularizar redes. O mecanismo delas é o mesmo?

a) Sim: ambas desligam neurônios aleatoriamente;
b) Não: Dropout zera saídas temporariamente, enquanto L2 penaliza pesos grandes;
c) Sim: ambas removem pesos permanentemente;
d) Não: L2 só funciona durante a previsão.

Pergunta: L2 faz a mesma coisa que Dropout?

Para discutir

As duas técnicas são usadas para regularizar redes. O mecanismo delas é o mesmo?

Resposta: b

Dropout altera temporariamente as ativações durante o treino. **L2** adiciona uma penalidade à função de perda.

Podemos usar as duas no mesmo modelo, mas isso não garante resultados melhores.

Pergunta: regularizar mais é sempre melhor?

Para discutir

Aumentamos bastante Dropout e L2. Depois, a rede passou a errar **mais no treino e na validação**. O que pode ter acontecido?

a) A regularização ficou forte demais;
b) Isso prova que há mais sobreajuste;
c) Precisamos sempre aumentar ainda mais L2;
d) O teste deve substituir a validação.

Pergunta: regularizar mais é sempre melhor?

Para discutir

Aumentamos bastante Dropout e L2. Depois, a rede passou a errar **mais no treino e na validação**. O que pode ter acontecido?

Resposta: a

Regularização excessiva pode gerar **subajuste**: a rede tem dificuldade até para aprender padrões relevantes.

Compare treino e validação e ajuste a intensidade da regularização com cuidado.

Aumentação de dados

Para discutir: mais fotos, sem fotografar

Para discutir

Vocês têm 500 fotos de placas, já rotuladas. Como conseguir mais fotos para o treino, **sem fotografar** nem rotular mais nada?

Para discutir: mais fotos, sem fotografar

Para discutir

Vocês têm 500 fotos de placas, já rotuladas. Como conseguir mais fotos para o treino, **sem fotografar** nem rotular mais nada?

- Criar **variações** das fotos que já temos: girar um pouco, escurecer, aproximar, deslocar;
- O rótulo continua o mesmo: uma placa *ABC1D23* um pouco girada continua sendo *ABC1D23*;
- É a **aumentação de dados** (*data augmentation*).

Aumentação de dados: a mesma placa, várias fotos



original

Aumentação de dados: a mesma placa, várias fotos



original



girada 12°

Aumentação de dados: a mesma placa, várias fotos



original



girada 12°



mais escura (noite)

Aumentação de dados: a mesma placa, várias fotos



original



girada 12°



mais escura (noite)



aproximada (zoom)

Aumentação de dados: a mesma placa, várias fotos



original



girada 12°



mais escura (noite)



aproximada (zoom)



deslocada

Aumentação de dados: a mesma placa, várias fotos



original



girada 12°



mais escura (noite)



aproximada (zoom)



deslocada



borrada (veículo andando)

Aumentação de dados: a mesma placa, várias fotos



original



girada 12°



mais escura (noite)



aproximada (zoom)



deslocada



borrada (veículo andando)



com ruído (câmera ruim)

Aumentação de dados: a mesma placa, várias fotos



original



girada 12°



mais escura (noite)



aproximada (zoom)



deslocada



borrada (veículo andando)



com ruído (câmera ruim)



mais clara (sol)

A cada época, o Keras sorteia uma variação nova de cada foto. As variações aleatórias dificultam a memorização e podem melhorar o desempenho em imagens novas.

Palpite: espelhar a placa?



espelhada



de cabeça para baixo

Palpite, antes de ver o resultado

Uma rede vai **ler os caracteres** das placas.

Essas duas variações ajudam no treino?

Palpite: espelhar a placa?



espelhada



de cabeça para baixo

Palpite, antes de ver o resultado

Uma rede vai **ler os caracteres** das placas. Essas duas variações ajudam no treino?

- **Não**. Placas espelhadas e de cabeça para baixo **não aparecem** nas câmeras: a rede gastaria esforço com algo que nunca vai ver;
- E pode ensinar errado: espelhado, um *b* vira *d*, e de cabeça para baixo, um *6* vira *9*;

Palpite: espelhar a placa?



espelhada



de cabeça para baixo

Palpite, antes de ver o resultado

Uma rede vai **ler os caracteres** das placas. Essas duas variações ajudam no treino?

- **Não**. Placas espelhadas e de cabeça para baixo **não aparecem** nas câmeras: a rede gastaria esforço com algo que nunca vai ver;
- E pode ensinar errado: espelhado, um *b* vira *d*, e de cabeça para baixo, um *6* vira *9*;
- Para outra tarefa, “**há um veículo na imagem?**”, espelhar é ótimo: um carro espelhado continua sendo um carro.

Palpite: o 6 de cabeça para baixo



um 6

Palpite, antes de ver o resultado

Para dígitos escritos à mão, girar a imagem 180° é uma boa aumentação?

Palpite: o 6 de cabeça para baixo



um 6 | o 6, girado | um 9

Palpite, antes de ver o resultado

Para dígitos escritos à mão, girar a imagem 180° é uma boa aumentação?

Não: o 6 girado **parece um 9**, e iria para o treino com o rótulo 6. A aumentação ensinaria errado. Girar **um pouco** (até uns 15°) serve: um 6 inclinado continua sendo 6.

Palpite: o 6 de cabeça para baixo



um 6



o 6, girado



um 9

Palpite, antes de ver o resultado

Para dígitos escritos à mão, girar a imagem 180° é uma boa aumentação?

Não: o 6 girado **parece um 9**, e iria para o treino com o rótulo 6. A aumentação ensinaria errado. Girar **um pouco** (até uns 15°) serve: um 6 inclinado continua sendo 6.

A regra da aumentação

Só vale uma variação que (1) **poderia aparecer** nos dados reais e (2) **não muda a resposta certa**.

Atividade: qual aumentação faz sentido?

Em grupos, 5 min. Para cada tarefa: a variação ajuda (sim) ou atrapalha (não)?

Tarefa	Espelhar	Girar pouco	Girar 180°	Brilho
1. Há um veículo na imagem da câmera?				
2. Ler os caracteres da placa				
3. Dígito escrito à mão				
4. Foto de drone: há uma construção?				

Atividade: qual aumentação faz sentido?

Em grupos, 5 min. Para cada tarefa: a variação ajuda (sim) ou atrapalha (não)?

Tarefa	Espelhar	Girar pouco	Girar 180°	Brilho
1. Há um veículo na imagem da câmera?	sim	sim	não	sim
2. Ler os caracteres da placa				
3. Dígito escrito à mão				
4. Foto de drone: há uma construção?				

Atividade: qual aumentação faz sentido?

Em grupos, 5 min. Para cada tarefa: a variação ajuda (sim) ou atrapalha (não)?

Tarefa	Espelhar	Girar pouco	Girar 180°	Brilho
1. Há um veículo na imagem da câmera?	sim	sim	não	sim
2. Ler os caracteres da placa	não	sim	não	sim
3. Dígito escrito à mão				
4. Foto de drone: há uma construção?				

Atividade: qual aumentação faz sentido?

Em grupos, 5 min. Para cada tarefa: a variação ajuda (sim) ou atrapalha (não)?

Tarefa	Espelhar	Girar pouco	Girar 180°	Brilho
1. Há um veículo na imagem da câmera?	sim	sim	não	sim
2. Ler os caracteres da placa	não	sim	não	sim
3. Dígito escrito à mão	não	sim	não	sim
4. Foto de drone: há uma construção?				

Atividade: qual aumentação faz sentido?

Em grupos, 5 min. Para cada tarefa: a variação ajuda (sim) ou atrapalha (não)?

Tarefa	Espelhar	Girar pouco	Girar 180°	Brilho
1. Há um veículo na imagem da câmera?	sim	sim	não	sim
2. Ler os caracteres da placa	não	sim	não	sim
3. Dígito escrito à mão	não	sim	não	sim
4. Foto de drone: há uma construção?	sim	sim	sim	sim

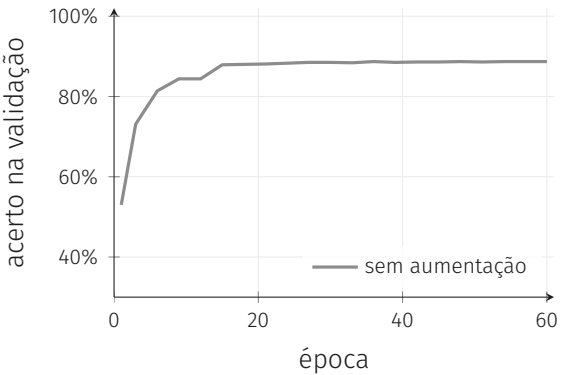
- Tarefa 1: câmeras não ficam de cabeça para baixo. Girar 180° não ensina errado, mas é esforço perdido;
- Tarefa 4: vista de cima, não existe “para cima”. Todas as rotações aparecem nos dados reais.

No Keras: camadas de aumentação

```
aumentacao = keras.Sequential([
    layers.RandomFlip("horizontal"),    # espelha (so se fizer sentido!)
    layers.RandomRotation(0.05),        # gira ate 5% de uma volta (18 graus)
    layers.RandomZoom(0.1),             # aproxima ou afasta ate 10%
    layers.RandomTranslation(0.1, 0.1),# desloca ate 10%
    layers.RandomBrightness(0.2),       # brilho; pixels de 0 a 255
])
modelo = keras.Sequential([
    keras.Input(shape=(240, 500, 3)),
    aumentacao,                         # a primeira "camada" da rede
    layers.Conv2D(32, 3, activation="relu"),
    # ... o resto da CNN
])
```

A aumentação funciona **só no treino** (`fit`). No `evaluate` e no `predict`, a imagem passa como está.

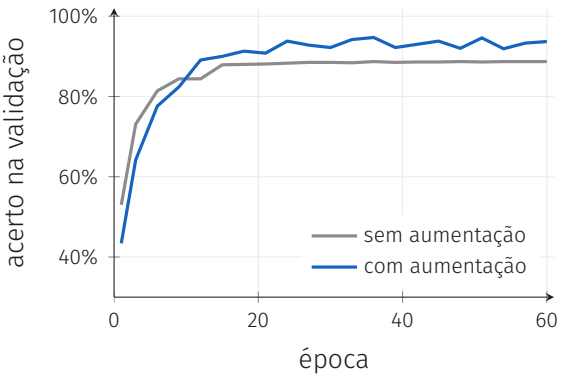
Resultado: 500 imagens, com e sem aumentação



- As mesmas 500 imagens. Aumentação: girar até 18°, aproximar até 10%, deslocar até 10%;

Nosso experimento. Com as 60.000 imagens e sem aumentação, a mesma CNN chega a 98,9%.

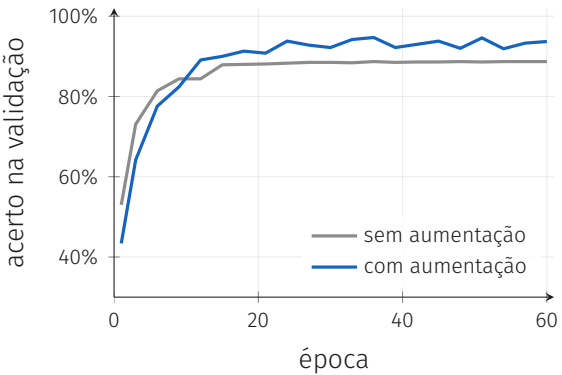
Resultado: 500 imagens, com e sem aumentação



- As mesmas 500 imagens. Aumentação: girar até 18°, aproximar até 10%, deslocar até 10%;
- Com aumentação, a validação **continua subindo** depois que a outra parou;

Nosso experimento. Com as 60.000 imagens e sem aumentação, a mesma CNN chega a 98,9%.

Resultado: 500 imagens, com e sem aumentação



- As mesmas 500 imagens. Aumentação: girar até 18°, aproximar até 10%, deslocar até 10%;
- Com aumentação, a validação **continua subindo** depois que a outra parou;
- No teste (8.000 dígitos novos):

	Acerto	Erros
sem	91,5%	676
com	95,0%	402

Nosso experimento. Com as 60.000 imagens e sem aumentação, a mesma CNN chega a 98,9%.

Palpite: e o acerto no treino?

Palpite, antes de ver o resultado

Sem aumentação, o acerto no treino chegou a 100%. Com aumentação, ele fica maior, igual ou menor?

Palpite: e o acerto no treino?

Palpite, antes de ver o resultado

Sem aumentação, o acerto no treino chegou a 100%. Com aumentação, ele fica maior, igual ou menor?

500 imagens, época 60	Treino	Validação
sem aumentação	100,0%	88,7%
com aumentação	97,6%	93,7%

- **Menor**: o treino ficou mais difícil, porque as imagens mudam a cada época e não dá para decorar;
- Com aumentação, o acerto na validação aumentou de 88,7% para 93,7%: melhor generalização.

Deslocamentos: densa, CNN e CNN com aumentação

Treinamos com os dígitos do MNIST, centralizados. No teste, deslocamos cada dígito alguns pixels para a direita.

Deslocamento no teste	0	2 pixels	4 pixels	6 pixels
Densa	97,3%	78,7%	32,7%	12,4%

Nosso experimento: 5 épocas com as 60.000 imagens; teste com 10.000.

Deslocamentos: densa, CNN e CNN com aumentação

Treinamos com os dígitos do MNIST, centralizados. No teste, deslocamos cada dígito alguns pixels para a direita.

Deslocamento no teste	0	2 pixels	4 pixels	6 pixels
Densa	97,3%	78,7%	32,7%	12,4%
CNN	98,9%	96,6%	67,0%	27,4%

- A CNN aguenta melhor os deslocamentos pequenos (o filtro e o pooling), mas **não os grandes**: a camada densa do fim ainda depende da posição;

Nosso experimento: 5 épocas com as 60.000 imagens; teste com 10.000.

Deslocamentos: densa, CNN e CNN com aumentação

Treinamos com os dígitos do MNIST, centralizados. No teste, deslocamos cada dígito alguns pixels para a direita.

Deslocamento no teste	0	2 pixels	4 pixels	6 pixels
Densa	97,3%	78,7%	32,7%	12,4%
CNN	98,9%	96,6%	67,0%	27,4%
CNN treinada com deslocamentos	94,4%	94,5%	94,4%	91,8%

- A CNN aguenta melhor os deslocamentos pequenos (o filtro e o pooling), mas **não os grandes**: a camada densa do fim ainda depende da posição;
- Treinada com deslocamentos aleatórios de até 7 pixels (`RandomTranslation`), a CNN fica mais robusta a mudanças de posição semelhantes às vistas no treino.

Nosso experimento: 5 épocas com as 60.000 imagens; teste com 10.000.

Palpite: o preço da aumentação

Palpite, antes de ver o resultado

Com os dígitos **centralizados**, a CNN comum acertou 98,9%, e a CNN treinada com deslocamentos, 94,4%. Por que ela piorou justamente no caso mais fácil?

Palpite: o preço da aumentação

Palpite, antes de ver o resultado

Com os dígitos **centralizados**, a CNN comum acertou 98,9%, e a CNN treinada com deslocamentos, 94,4%. Por que ela piorou justamente no caso mais fácil?

- O treino ficou **bem mais difícil**: dígitos deslocados até 7 pixels, às vezes cortados na borda. Em só 5 épocas, ela aprendeu menos sobre cada dígito;
- É uma **troca**: um pouco menos no caso fácil, muito mais nos difíceis (91,8% contra 27,4% com 6 pixels);

Palpite: o preço da aumentação

Palpite, antes de ver o resultado

Com os dígitos **centralizados**, a CNN comum acertou 98,9%, e a CNN treinada com deslocamentos, 94,4%. Por que ela piorou justamente no caso mais fácil?

- O treino ficou **bem mais difícil**: dígitos deslocados até 7 pixels, às vezes cortados na borda. Em só 5 épocas, ela aprendeu menos sobre cada dígito;
- É uma **troca**: um pouco menos no caso fácil, muito mais nos difíceis (91,8% contra 27,4% com 6 pixels);
- Vale a regra da aumentação: o que aparece no uso real? Se os dígitos sempre chegam centralizados, deslocar tanto não compensa. Se aparecem deslocados **dentro do recorte**, pode ajudar. Encontrar a placa numa foto inteira exige **localização ou detecção**.

Pergunta: quando aumentar?

Um colega pegou as 500 fotos, criou 10 variações de cada uma (5.000 fotos), e **depois** dividiu em treino e teste. O acerto no teste foi 99%. O que há de errado?

a) Nada: mais fotos, mais acerto;
b) Dez variações é pouco;
c) Variações da mesma foto caíram no treino e no teste: é vazamento;
d) A aumentação não pode ser feita antes do treino.

Pergunta: quando aumentar?

Um colega pegou as 500 fotos, criou 10 variações de cada uma (5.000 fotos), e **depois** dividiu em treino e teste. O acerto no teste foi 99%. O que há de errado?

a) Nada: mais fotos, mais acerto;
b) Dez variações é pouco;
c) Variações da mesma foto caíram no treino e no teste: é vazamento;
d) A aumentação não pode ser feita antes do treino.

Resposta: c. O teste tem "quase cópias" das fotos de treino: a rede reconhece a foto, e não a placa. São as duplicatas entre os conjuntos da Aula 2. Primeiro dividam, depois aumentem, e **só o treino**. A aumentação dentro do modelo, como no Keras, já faz isso.

Transferência de aprendizado

800 fotos de veículos: começar do zero?

Para discutir

Queremos distinguir **carro, moto, caminhão e ônibus**, mas temos só 800 fotos rotuladas. Vale treinar todos os filtros de uma CNN do zero?

CNN do zero

Os filtros começam sem conhecimento das imagens. Precisamos aprender **todas** as características com nossos exemplos.

Rede pré-treinada

Reaproveitamos filtros aprendidos em muitas fotos e treinamos **uma nova saída** para nossas classes.

800 fotos de veículos: começar do zero?

Para discutir

Queremos distinguir **carro, moto, caminhão e ônibus**, mas temos só 800 fotos rotuladas. Vale treinar todos os filtros de uma CNN do zero?

CNN do zero

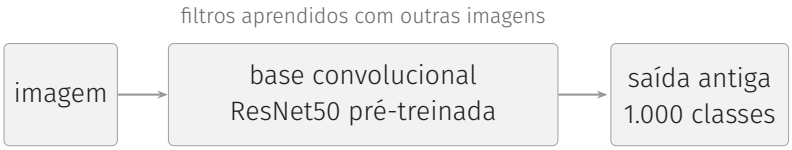
Os filtros começam sem conhecimento das imagens. Precisamos aprender **todas** as características com nossos exemplos.

Rede pré-treinada

Reaproveitamos filtros aprendidos em muitas fotos e treinamos **uma nova saída** para nossas classes.

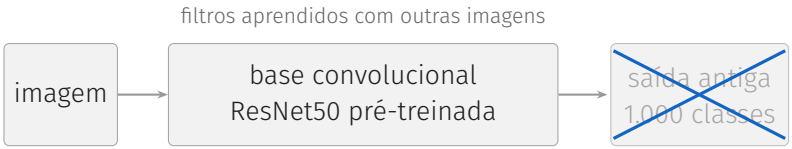
Bom ponto de partida: experimentar uma base pré-treinada, pois veículos comuns são próximos de imagens do ImageNet. Em câmeras **térmicas**, a diferença de modalidade exige ainda mais testes: a transferência pode não ajudar.

Transferência de aprendizado, passo a passo



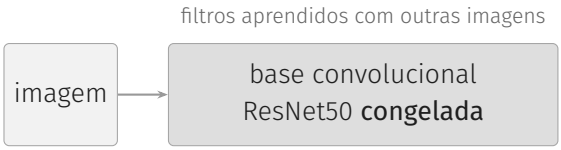
1. Carregar a ResNet50 treinada no **ImageNet**;

Transferência de aprendizado, passo a passo



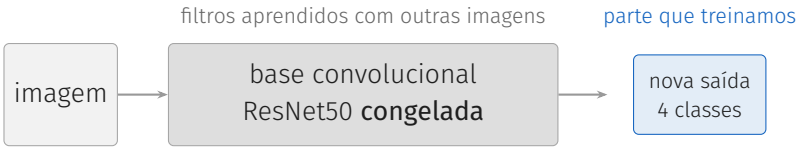
1. Carregar a ResNet50 treinada no **ImageNet**;
2. Retirar a saída das 1.000 classes antigas;

Transferência de aprendizado, passo a passo



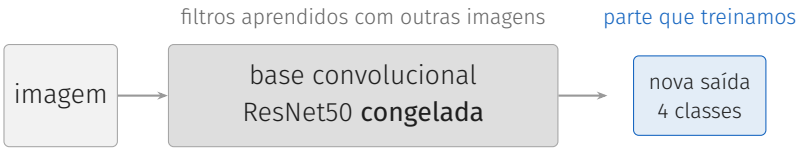
1. Carregar a ResNet50 treinada no **ImageNet**;
2. Retirar a saída das 1.000 classes antigas;
3. **Congelar** os pesos da base: eles não mudam;

Transferência de aprendizado, passo a passo



1. Carregar a ResNet50 treinada no **ImageNet**;
2. Retirar a saída das 1.000 classes antigas;
3. **Congelar** os pesos da base: eles não mudam;
4. Adicionar uma nova saída de **4 classes** e treiná-la com nossas fotos;

Transferência de aprendizado, passo a passo



1. Carregar a ResNet50 treinada no **ImageNet**;
2. Retirar a saída das 1.000 classes antigas;
3. **Congelar** os pesos da base: eles não mudam;
4. Adicionar uma nova saída de **4 classes** e treiná-la com nossas fotos;
5. Se necessário, **ajustar parte da base** com uma taxa de aprendizado pequena.

Qual rede pré-treinada escolher?

Modelo no Keras	Ponto forte	Quando experimentar
`MobileNetV2`	Menor custo computacional	Quando tempo e memória importam
`EfficientNetB0`	Boa relação custo/desempenho	Como candidato inicial
`ResNet50`	Referência muito estudada	Para comparar com a rede da aula
`VGG16`	Arquitetura mais simples	Para comparar, apesar de ser pesada
`Xception`	Outra família de arquitetura	Para ampliar os experimentos

Regra prática

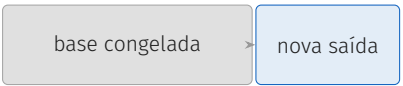
Comecem por **MobileNetV2, EfficientNetB0 e ResNet50**. Comparem **validação, memória e tempo**: a maior rede não é necessariamente a melhor.

Atenção: cada família pode exigir **pré-processamento próprio** para as imagens.

Modelos e preparo: keras.io/api/applications/.

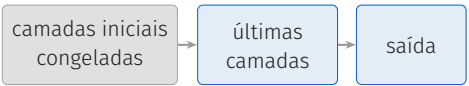
Congelar ou fazer ajuste fino?

Primeira etapa: base congelada



Treino mais simples e barato.
Primeiro passo recomendado.

Segunda etapa: ajuste fino



Mais parâmetros mudam.
Mais custo e risco de sobreajuste.

Quando tentar a segunda etapa?

Depois de treinar a nova saída. Se adaptar filtros **melhorar a validação**, pode valer a pena; caso contrário, mantenha a base congelada.

Não há garantia: **fine-tuning pode piorar** quando há poucos dados.

No Keras: carregar a base

```
import keras
from keras import layers, ops
from keras.applications import ResNet50

# (1) Pre-treinada: usa pesos do ImageNet
base = ResNet50(weights="imagenet",
                include_top=False, pooling="avg",
                input_shape=(224, 224, 3))
base.trainable = False

# (2) Entrada, com imagens RGB na escala 0..255
entrada = keras.Input(shape=(224, 224, 3))
x = layers.RandomFlip("horizontal")(entrada)
x = layers.RandomRotation(0.05)(x)

# (3) Pre-processamento especifico da ResNet50
x = ops.flip(x, axis=-1)  # RGB -> BGR
x = x - ops.convert_to_tensor([103.939, 116.779, 123.68])
```

Cuidado: MobileNetV2, EfficientNetB0 e outras famílias podem usar um preparo diferente.

No Keras: saída nova e treinamento

```
# (4) Aumentacao + base congelada + 4 classes
x = base(x, training=False)
saida = layers.Dense(4, activation="softmax")(x)
modelo = keras.Model(entrada, saida)

modelo.compile(optimizer="adam",
    loss="sparse_categorical_crossentropy", metrics=["accuracy"])

parar = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=3,
    restore_best_weights=True)

# (5) Treinar somente a nova Dense(4)
modelo.fit(X_train, y_train, epochs=30, batch_size=32,
    validation_data=(X_val, y_val), callbacks=[parar])
```

A base mantém os pesos do ImageNet; a nova saída aprende com as quatro classes.

`EarlyStopping` usa a validação, como vimos antes.

Quantos parâmetros realmente treinamos?

```
Layer (type)                 Output Shape              Param #
=================================================================
random_flip (RandomFlip)     (None, 224, 224, 3)             0
random_rotation (...)        (None, 224, 224, 3)             0
resnet50 (Functional)        (None, 2048)           23,587,712
dense (Dense)                (None, 4)                   8,196
=================================================================
Total params: 23,595,908
Trainable params: 8,196
Non-trainable params: 23,587,712
```

- O `pooling="avg"` resume cada um dos **2.048 mapas finais** por uma média;
- A saída aprende $(2.048 + 1) \times 4 = 8.196$ parâmetros;

Quantos parâmetros realmente treinamos?

```
Layer (type)                 Output Shape              Param #
=================================================================
random_flip (RandomFlip)     (None, 224, 224, 3)             0
random_rotation (...)        (None, 224, 224, 3)             0
resnet50 (Functional)        (None, 2048)           23,587,712
dense (Dense)                (None, 4)                   8,196
=================================================================
Total params: 23,595,908
Trainable params: 8,196
Non-trainable params: 23,587,712
```

- O `pooling="avg"` resume cada um dos **2.048 mapas finais** por uma média;
- A saída aprende $(2.048 + 1) \times 4 = 8.196$ parâmetros;
- Aproveitamos **23,6 milhões** de pesos já aprendidos, sem atualizá-los nesta etapa.

No Keras: ajuste fino, se necessário

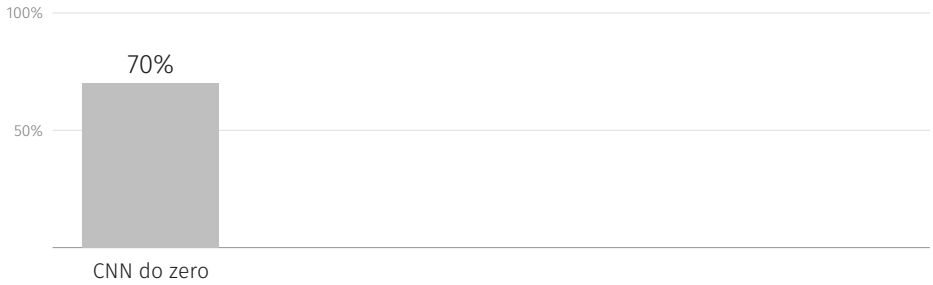
```
# So depois de treinar a nova saida
base.trainable = True
for camada in base.layers[:-20]:  # exemplo: liberar ultimas 20
    camada.trainable = False
for camada in base.layers:
    if isinstance(camada, layers.BatchNormalization):
        camada.trainable = False

# Taxa pequena e compilacao novamente
modelo.compile(optimizer=keras.optimizers.Adam(1e-5),
    loss="sparse_categorical_crossentropy", metrics=["accuracy"])
modelo.fit(X_train, y_train, epochs=10, batch_size=32,
    validation_data=(X_val, y_val), callbacks=[parar])
```

Atenção: 20 camadas é só um exemplo, não uma regra. `base(x, training=False)` preserva o comportamento de BatchNormalization. **Recompile** após mudar `trainable`.

O ajuste fino valeu a pena?

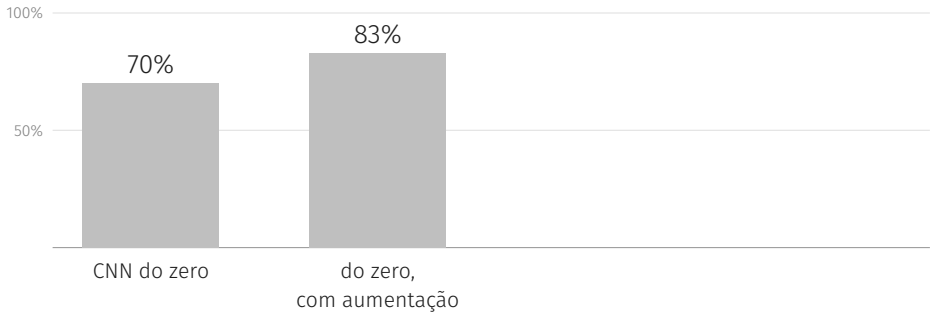
Exemplo de **gatos e cachorros**, com 2.000 fotos de treino:



Chollet, 2. ed., cap. 8: VGG16, números aproximados; NÃO são resultados da ResNet50 deste código.

O ajuste fino valeu a pena?

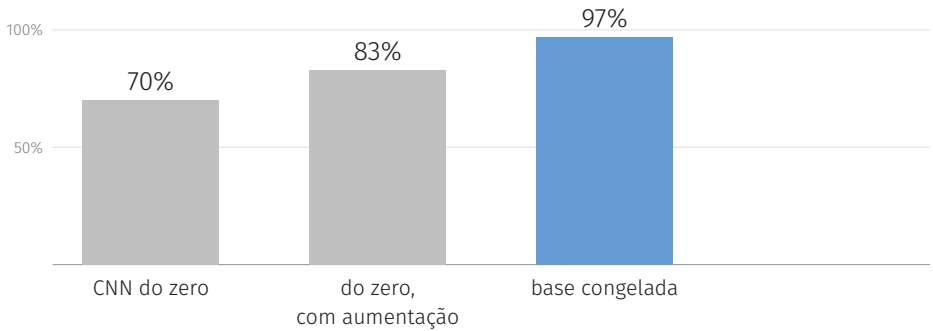
Exemplo de **gatos e cachorros**, com 2.000 fotos de treino:



Chollet, 2. ed., cap. 8: VGG16, números aproximados; NÃO são resultados da ResNet50 deste código.

O ajuste fino valeu a pena?

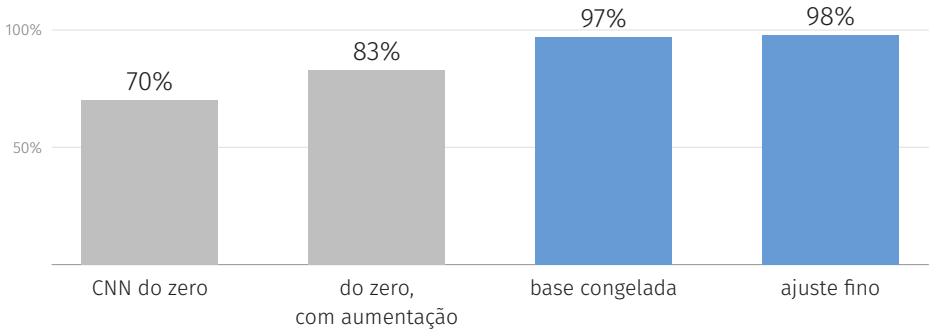
Exemplo de **gatos e cachorros**, com 2.000 fotos de treino:



Chollet, 2. ed., cap. 8: VGG16, números aproximados; NÃO são resultados da ResNet50 deste código.

O ajuste fino valeu a pena?

Exemplo de **gatos e cachorros**, com 2.000 fotos de treino:



Neste caso, a maior melhoria vem da base pré-treinada. O ganho extra do ajuste fino é pequeno: vale o custo? Decidam usando validação.

Chollet, 2. ed., cap. 8: VGG16, números aproximados; NÃO são resultados da ResNet50 deste código.

Interpretabilidade

A CNN acertou. Mas aprendeu o motivo certo?

Para discutir

Uma CNN acerta que a foto contém um **cachorro**. Será que reconheceu o animal ou aprendeu a associar a classe a **um fundo ou uma marca na imagem**?

Previsão

O que saiu?

Exemplo: “cachorro”.

Investigação

Que pistas influenciaram?

Objetos, fundo, marcas ou textura?

A CNN acertou. Mas aprendeu o motivo certo?

Para discutir

Uma CNN acerta que a foto contém um **cachorro**. Será que reconheceu o animal ou aprendeu a associar a classe a **um fundo ou uma marca na imagem**?

Previsão

O que saiu?

Exemplo: “cachorro”.

Investigação

Que pistas influenciaram?

Objetos, fundo, marcas ou textura?

Lembrem do exemplo do **husky e da neve** (Aula 2): prever corretamente não garante que a rede usou o critério esperado.

Grad-CAM: onde há pistas para cada classe?

foto → CNN → classe escolhida → mapa Grad-CAM

Mesma imagem



Classe “cachorro”



Classe “gato”



Para discutir

É a **mesma foto**. Por que o mapa muda quando escolhemos outra classe?

Regiões mais intensas recebem maior atribuição **para a classe escolhida**. O Grad-CAM usa ativações e gradientes da CNN; **não** delimita objetos nem prova causas.

Mapas reais de CNN didática com recortes da ilustração; não são resultados da ResNet50/Xception.

Grad-CAM na prática: como usar no Keras?

Precisamos de uma CNN treinada, uma imagem e uma classe a investigar.

Exemplo: ResNet50 pré-treinada

```
rede = keras.applications.ResNet50(
    weights="imagenet")
foto = keras.utils.load_img(
    "foto.jpg", target_size=(224, 224))
x = keras.utils.img_to_array(foto)[None]
x = keras.applications.resnet50.preprocess_input(x)
classe = rede.predict(x, verbose=0).argmax()
mapa = gradcam(rede, x, classe,
    "conv5_block3_out")
sobrepor(foto, mapa).save("resultado.png")
```

O que fazemos?

1. Preparamos a foto **como a rede espera**;
2. Escolhemos a **classe** (aqui, a previsão);
3. Calculamos o mapa a partir de uma **camada convolucional**;
4. Sobrepomos o mapa à **foto original**.

Importante: *gradcam* e *sobrepor* são funções auxiliares, **não comandos nativos** do Keras.

Experimento: ensinamos uma pista falsa à CNN

Objetivo: descobrir se a rede consegue acertar **sem precisar olhar o animal**.



Cachorro + vermelho



Gato + azul

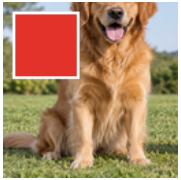
Como construímos os dados de treino?

1. Partimos de **dois recortes de uma ilustração**: um gato e um cachorro;
2. Criamos **130 variações por classe** (recorte, rotação, desfoque etc.);
3. Treinamos a CNN com **vermelho sempre em cachorro** e **azul sempre em gato**.

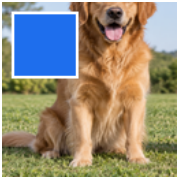
Armadilha: no treino, a cor do quadrado prevê o rótulo perfeitamente, mesmo sem reconhecer pelos, focinho ou orelhas.

Como testar se a rede usou esse atalho?

Criamos duas versões da mesma imagem de cachorro:



Cor vista no treino



Cor associada a gatos

Intervenção controlada

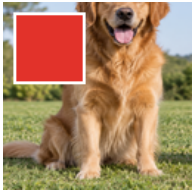
Mantemos **o mesmo cachorro e o mesmo fundo**. Mudamos **somente a cor do quadrado** e medimos novamente a saída da CNN para “cachorro”.

Palpite, antes de ver o resultado

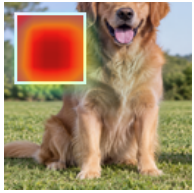
Se a CNN reconheceu o cachorro, trocar só a cor deveria fazê-la prever “gato?

Resultado: a previsão mudou com o quadrado

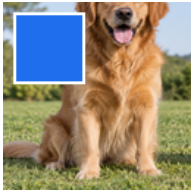
Mesma foto e mesma CNN; mapas para a classe “cachorro”:



Quadrado vermelho



Grad-CAM “cachorro”



Quadrado azul



Grad-CAM “cachorro”

Saída softmax para “cachorro”

Vermelho: 99,99% $\longrightarrow$ Azul: 41,66%

Interpretação: com o quadrado azul, a CNN chegou a prever **“gato”** para o mesmo cachorro. O mapa destaca regiões suspeitas; trocar apenas a pista mostra que ela afeta a decisão.

Experimento didático.

O que um Grad-CAM não consegue provar?

Para discutir

Se o mapa destaca o **rosto do cachorro**, podemos concluir que o modelo **não** está usando o quadrado colorido?

- **Não.** A cor do mapa não é probabilidade por pixel, nem uma medida causal exata;
- Um mapa pode omitir pistas: a resolução da camada e o método influenciam o resultado;
- Para testar hipóteses, podemos **trocar ou ocultar regiões** e medir como muda a pontuação; ocultar também pode criar fotos artificiais;
- Uma avaliação responsável reúne **métricas em dados novos, análise de erros e mapas de atribuição**.

Para levar ao projeto: o modelo continua acertando quando muda a câmera, o cenário ou uma pista visual que **não deveria importar**?

Aplicações e fechamento

Estudos de caso

Para discutir

Para cada caso, escolham uma estratégia e justifiquem em **5 minutos**.

Situação	O que precisam decidir?
800 fotos de veículos	Treinar do zero ou aproveitar uma base? Que aumentação faria sentido?
Detecção de arma: só 2% positivos	Acurácia basta? Que erros preocupam mais?
Uma câmera nova, de outro bairro	Como testar robustez? Que pistas investigar com Grad-CAM?

Estudos de caso

Para discutir

Para cada caso, escolham uma estratégia e justifiquem em **5 minutos**.

Situação	O que precisam decidir?
800 fotos de veículos	Treinar do zero ou aproveitar uma base? Que aumentação faria sentido?
Detecção de arma: só 2% positivos	Acurácia basta? Que erros preocupam mais?
Uma câmera nova, de outro bairro	Como testar robustez? Que pistas investigar com Grad-CAM?

Não há uma receita universal. Divisão dos dados, métricas e custo dos erros fazem parte da escolha.

No projeto de vocês

1. Há **sobreajuste**? Comparem treino e validação;
2. As **aumentações** simulam situações reais sem mudar o rótulo?
3. Uma **base pré-treinada** combina com o tipo de imagem que vocês têm?
4. A validação contém fotos de **câmeras, datas ou locais** não vistos no treino?
5. Os erros e os mapas de atribuição sugerem algum **atalho**?

O que levar da Aula 5

1. **Treino bom não garante generalização.** Regularização tenta diminuir o sobreajuste;
2. **Dropout, L2, EarlyStopping** são ferramentas diferentes; avaliem seu efeito;
3. **Aumentação**: novas variações plausíveis, apenas no treino e depois de separar os dados;
4. **Transferência**: reutilizar uma base pré-treinada; congelar e, se necessário, ajustar com cuidado;
5. **Grad-CAM**: pista visual sobre uma previsão, não uma explicação causal definitiva.

Uma boa CNN não é apenas a que acerta: é a que generaliza e pode ser avaliada criticamente.

Referências

- CHOLLET, F. *Deep Learning with Python*. 2. ed. Manning, 2021. Caps. 8 e 9.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016. Caps. 7 e 9.
- SRIVASTAVA, N. et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 2014.
- HE, K. et al. Deep Residual Learning for Image Recognition. *CVPR*, 2016.
- SELVARAJU, R. R. et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV*, 2017.
- ZEILER, M. D.; FERGUS, R. Visualizing and Understanding Convolutional Networks. *ECCV*, 2014.
- *Keras documentation*: Transfer learning & fine-tuning; Grad-CAM class activation visualization.